



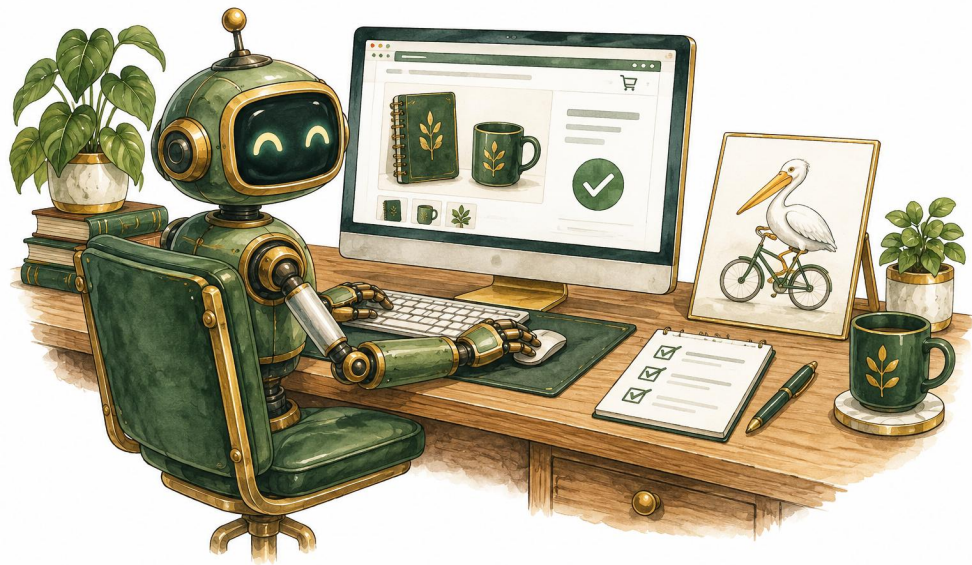
香港大學

THE UNIVERSITY OF HONG KONG

Post-training

COMP7607A · LECTURE 3

Learning to Follow Instructions and
Complete Tasks





Lecture outline

Why does a pre-trained model need further practice?

01 Overview	Why train after pre-training?
02 SFT	Learn from demonstrations
03 Reasoning	Why useful intermediate work helps
04 RL	Learn from outcomes and check progress
05 Hands-on	NanoChat + TinyZero
06 Insights	What frontier results really show

Working software, checked information and useful visual deliverables.



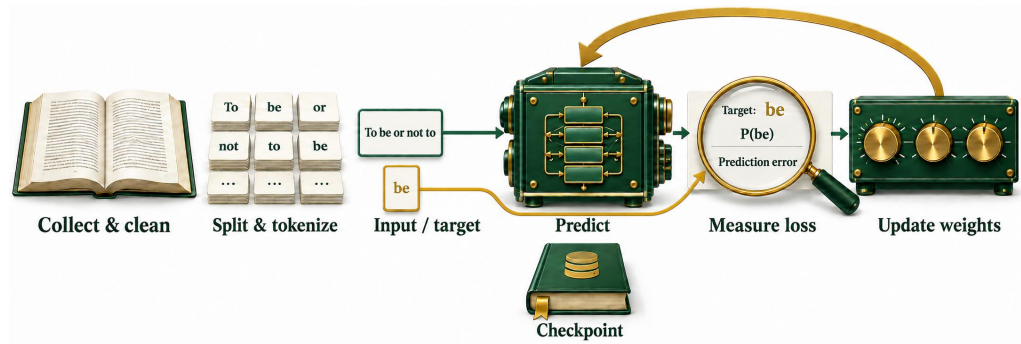
01

Why post-training?

SO FAR

We trained a model to predict text.

PART 1 OF 6



OUR NEXT QUESTION

What should it learn to follow user's instructions?



Recall: our next-token model

Text goes in. Scores for possible next tokens come out.

PREFIX	To be, or not to	NEXT TOKEN	be
--------	------------------	------------	----

INPUT

To be,
or not to

MODEL

Transformer
?

OUTPUT

be
one possible
next token

Lecture 2 followed this sentence through prediction and learning.



Text supplies its own answers

Hide the next token and use it as the answer.

To be, or not to be

What the model sees

To be, or not to

Answer in the text

be

The next token in the text supplies the training target.

A training step checks the prediction

The text says “be”. Compare the model’s guess with that answer.



Repeated prediction errors tell us how to update the weights.



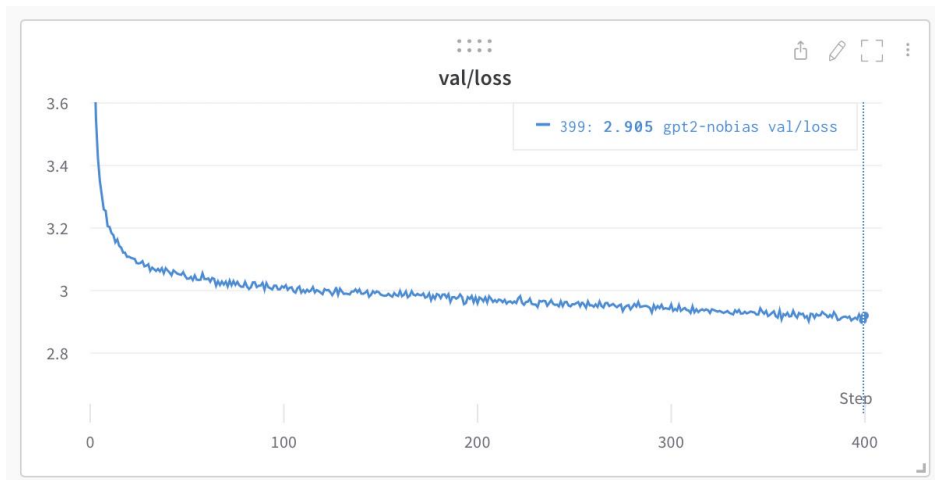
We improved prediction on unseen text

The validation loss falls as this model learns.

NanoGPT

GPT-2 (124M)

Validation on
OpenWebText



Lower validation loss checks prediction. What checks a completed user task?

One model can support different requests

Instructions turn different tasks into text input and text output.



Pre-training supplies a foundation. Reliable task behaviour needs targeted practice.



Read a lot. Ready to help?

Knowledge is a starting point.
What should it do with a request?





The request changes what counts as success

“Explain ‘To be, or not to be’ in one short sentence.”

Text continuation

“That is the question...”

Pre-training target: the next text

Following the instruction

“He is weighing whether to live or die.”

User goal: a short explanation

Post-training practises responses that satisfy the request.



Can the model finish a real flight search?

“Hong Kong to New York
tomorrow.
Compare fares including one bag.”



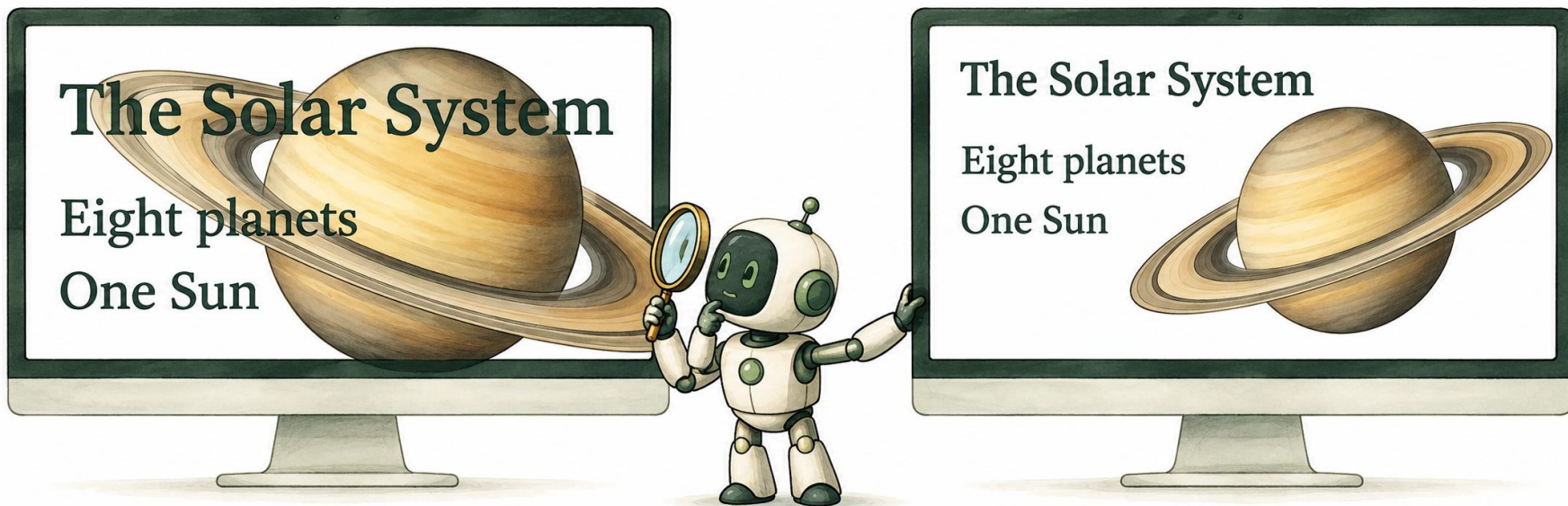


One bag changes the total

The assistant needs to check the baggage fee.



Can an agent check its own slides?





A saved PPT can still look wrong

The file opens

✓ slides.pptx

The slide needs checking

Right image?
Readable text?
Any overlap?

Render the slide and inspect what students will see.

Post-training changes the practice

We keep the learned foundation and practise useful behaviour.



SFT: learn a checked example

RL: learn from scored attempts

These learning methods can support coding, tool use and visual tasks.



SFT fits tasks with clear demonstrations

“Two apples, please.”



Order: apple × 2

“Hong Kong to JFK.”



search_flights(...)

**“Show tomorrow’s
meetings.”**



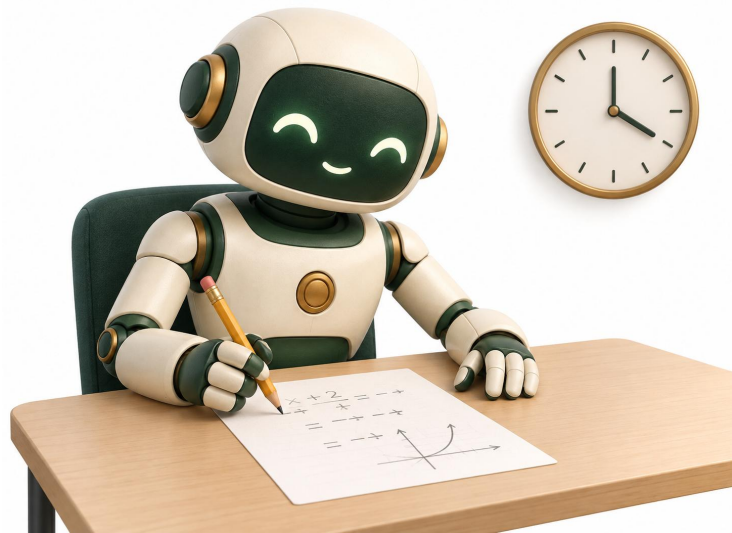
get_calendar(...)

Checked examples show what the model should output.



RL learns from scored attempts

\$12/hour
50 minutes
How much pay?



Try a solution → check the answer → learn from the score.



香港大學
THE UNIVERSITY OF HONG KONG

LLM FOUNDATION · LECTURE 3

02

SFT

Supervised fine-tuning

SO FAR

We have useful knowledge, but task behaviour needs practice.

PART 2 OF 6



OUR NEXT QUESTION

Could a checked example show the model what to do?



SFT adapts a pre-trained model

Already learned

**Language patterns
and general
knowledge**



Now practise

**Customer request
Correct order**

Pre-trained weights + task demonstrations = SFT



One method, several concrete tasks

“Two apples, please.”



Order: apple × 2

“Wake me at 7:30 a.m.”



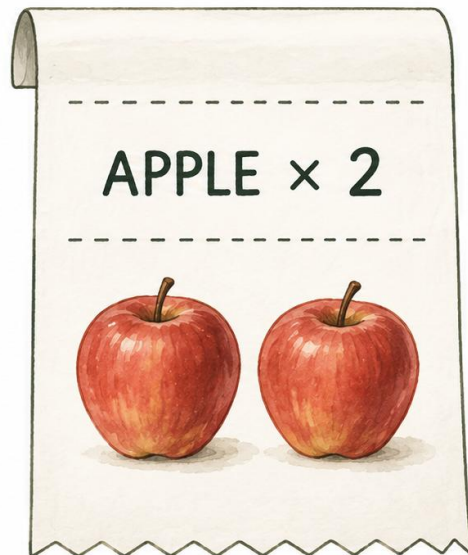
set_alarm("07:30")

“Find a flight to JFK.”



search_flights(...)

Teach a robot to write an order





Different words, precise orders

“Two apples, please.”



APPLE × 2

“Give me three bananas.”



BANANA × 3

“Just one apple.”



APPLE × 1



A flight search can be a training example

“Hong Kong to JFK, 1 October 2026.”

```
search_flights(  
    origin="HKG", destination="JFK",  
    date="2026-10-01")
```

The flight service returns current fares after the call.



The request supplies the time

**“Set an alarm
for 7:30 a.m.”**



```
set_alarm(  
    time="07:30")
```

07:30



A calendar lookup also needs the right date

“My meetings on 1 October 2026?”

```
get_calendar(date="2026-10-01")
```

The tool supplies the meetings. The model chooses the lookup.



Input and ground truth form a training pair

INPUT

“Two apples, please.”

GROUND TRUTH

```
{"item": "apple", "qty": 2}
```



The same pair in chat-training format

```
{"messages": [  
  {"role": "user",  
    "content": "Two apples, please."},  
  {"role": "assistant",  
    "content": "{\"item\": \"apple\", \"qty\": 2}"  
}]}
```



Flight search has a checkable ground truth

“Hong Kong to JFK, 1 October 2026.”

```
{"name": "search_flights",  
  "arguments": {"origin": "HKG",  
                "destination": "JFK",  
                "date": "2026-10-01"}}
```



The dataset is a collection of these pairs

Two apples. → {"item":"apple", "qty":2}

Three bananas. → {"item":"banana","qty":3}

Just one apple. → {"item":"apple", "qty":1}

Store both the customer's words and the checked answer.



The order is still generated as tokens

Input: “Two apples, please.”

{ "item" : "apple" , "qty" : 2 }

Simplified token boundaries

Predict the next token of the correct answer.



Training supplies the correct prefix

Already supplied

```
{ "item" :
```

→

"apple"

```
{ "item" : "apple" ,
```

→

"qty"

```
{ "item" : "apple" , "qty" :
```

→

2

Next target



Learn the answer tokens

Read the request

Two apples, please.

Score the answer

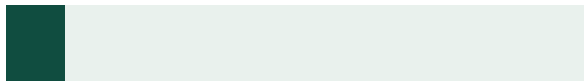
```
{ "item": "apple", "qty": 2 }
```



Loss asks: how likely was the correct token?

Correct next token: 2

Before learning



$$P(2) = 0.10$$

After learning



$$P(2) = 0.80$$

Higher probability for the correct token means lower loss.



A batch combines several training pairs

“Two apples, please.” $\rightarrow$ apple $\times$ 2

“Give me three bananas.” $\rightarrow$ banana $\times$ 3

“Just one apple.” $\rightarrow$ apple $\times$ 1

One batch: 3 pairs. Average the answer-token losses.



One loss averages many answer predictions

```
{ "item": "apple", "qty": 2 }
```

Every scored answer token contributes

Loss = average of $-\log P(\text{correct token})$



One SFT training step

```
inputs, targets, mask = prepare(record)
scores = model(inputs)
loss = next_token_loss(scores, targets, mask)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```



Did it learn orders, or just the examples?

Practised

“Two apples, please.”

APPLE × 2

Not practised

“Could I have four apples, thanks?”

?



Train, validation and test have different jobs

TRAIN

800 groups

Learn weights

VALIDATION

100 groups

Choose checkpoint

TEST

100 groups

Final report

Example split only. Keep the test set untouched until the end.



Split before making paraphrases

One family of requests

“Two apples.”

“Please give me two apples.”

“Apple, quantity two.”



ONE SPLIT

Group by conversation or template source, then split the groups.



Validation measures learning without changing it

Training batch

```
predict  
loss.backward()  
optimizer.step()
```

Validation batch

```
predict  
measure loss  
no weight update
```

Generation and the final test also keep weights fixed.



A new wording has a known correct answer

Input: “Could I have four apples, thanks?”

GROUND TRUTH

```
{"item": "apple", "qty": 4}
```



Model output versus ground truth

GROUND TRUTH

```
{"item":"apple",  
  "qty":4}
```

MODEL OUTPUT

```
{"item":"apple",  
  "qty":3}
```

Item matches. Quantity does not. This order is wrong.



Both fields must be correct

Four apples, please.

APPLE × 4

✓ Item and quantity

APPLE × 3

× Quantity

BANANA × 4

× Item



Parse the fields, then compare them

```
reply = model.generate(held_out_request)
order = parse_json(reply) # invalid output fails
item_ok = normalize(order.item) == "apple"
qty_ok = is_integer(order.qty) and order.qty == 4
correct = item_ok and qty_ok
```

Order accuracy = correct orders / all evaluated requests.



Loss and task accuracy answer different questions

Token loss

**How likely is the
reference answer?**

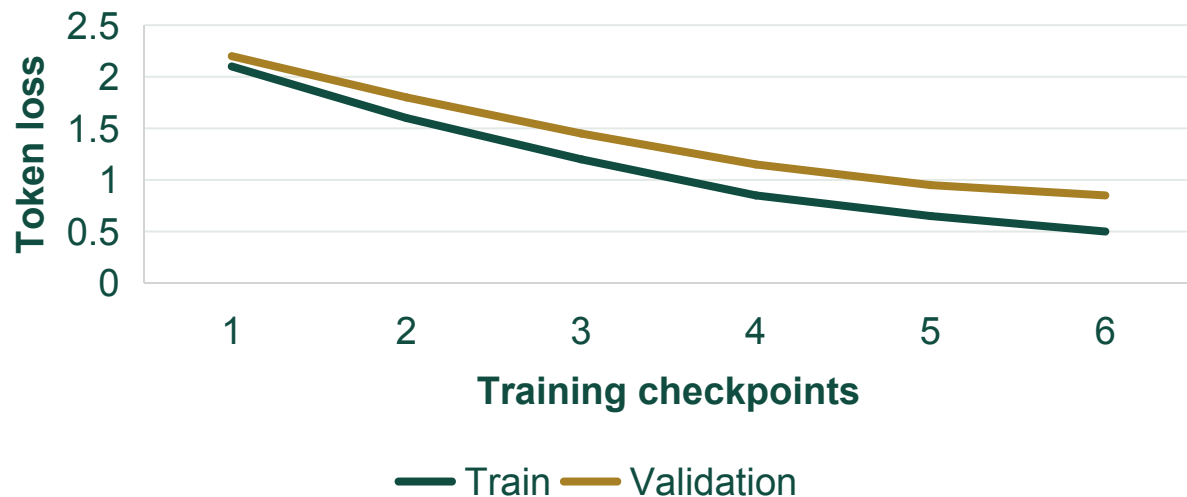
Order accuracy

**Did the generated
order get both fields
right?**

Track both on the same held-out requests.



Train and validation loss should both improve



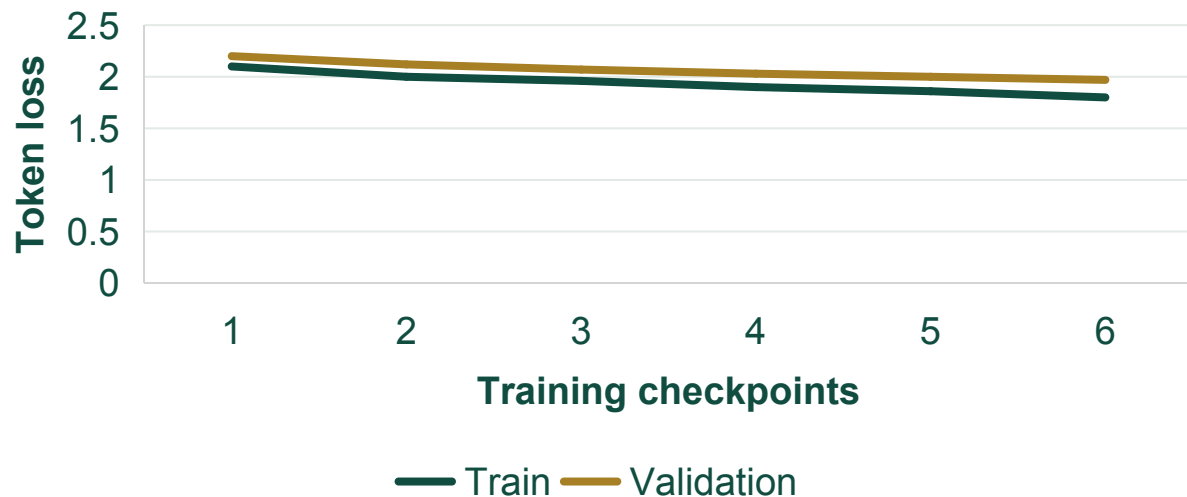
**Seen requests
get easier.**

**Unseen
requests
get easier too.**

Illustrative curves



Underfitting: even practice is difficult



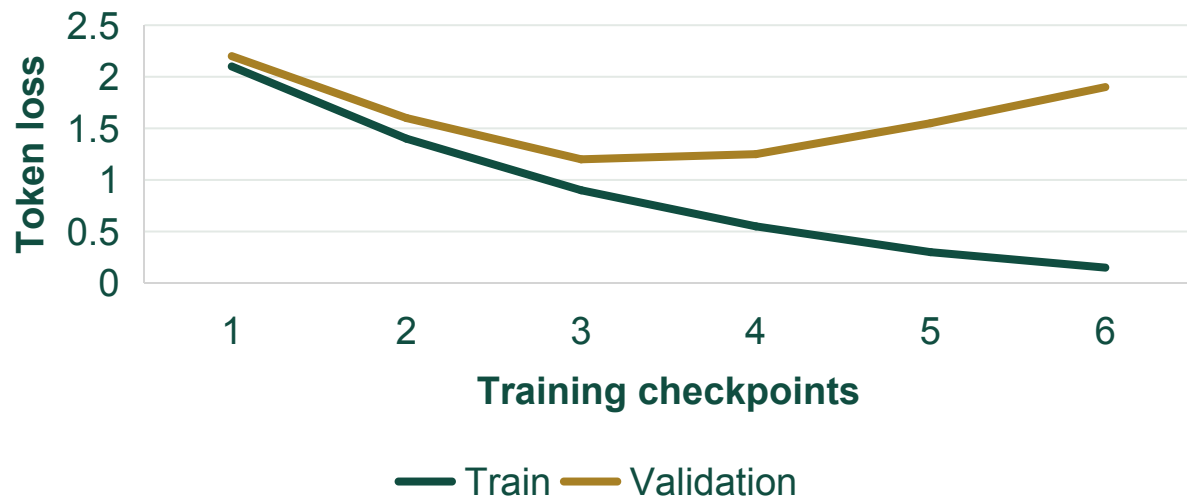
Train: high
Validation: high

Check examples
and training setup.

Illustrative curves



Overfitting: practice improves, new requests worsen



Train keeps falling.

Validation rises.

More practice
can stop helping.

Illustrative curves



Choose with validation, report with test

Checkpoint A

val loss 1.20

Checkpoint B

val loss 0.80

Checkpoint C

val loss 1.10

Select B



One final test

Illustrative validation values

A teacher model can create more examples





Start with a known order, vary the request

Known order

BANANA × 3



“Three bananas, please.”
“Can I get three bananas?”
“Bananas, three of them.”

Vary the wording while keeping the item and quantity fixed.



Check teacher samples before keeping them

Known order: banana × 3

Teacher's request

✓ Keep

“Three bananas, please.”

× Reject

“Two bananas, please.”

The generated wording must still match the known order.



The teacher also writes the target answer

“Can I get three bananas?”

CHECKED TEACHER OUTPUT

```
{"item": "banana", "qty": 3}
```

Teacher weights stay fixed. Student weights learn from the sample.



Generate, check, then train the student

```
for request in generated_train_requests:
    answer = teacher.generate(request)
    if matches_known_order(request, answer):
        train_set.add(request, answer)
student.sft(train_set)
```

Student SFT uses the same answer-token loss as before.



Good examples do not cover every situation

Learn a correct example



Try an unfamiliar request

What if completing the task needs several decisions?



03

Reasoning

SO FAR

Examples can teach useful responses. Some requests need several steps.

PART 3 OF 6



OUR NEXT QUESTION

What if the answer needs several useful steps?



Why write down the working?

Use 3, 4 and 5 once each.
Can you make 27?





The first combination misses the target

3 4 5

$$3 + 4 + 5 = 12$$

Target: 27

All three numbers appear, but the result is 12. The target is 27.



A useful intermediate step reveals a solution

Combine

$$4 + 5 = 9$$

Use the 3

$$9 \times 3 = 27$$

Answer

$$(4 + 5) \times 3$$

Check both conditions: the target is 27, and each number appears once.



Each new step becomes part of the input

REQUEST

Use 3, 4 and 5 to make 27.

WORK SO
FAR

$4 + 5 = 9$. Then $9 \times 3 =$



27

The prefix grows. The model predicts another token.

The model works through more steps. Its weights stay fixed.



SFT can teach a worked solution

INPUT

Use 3, 4 and 5 once each to make 27.

TARGET

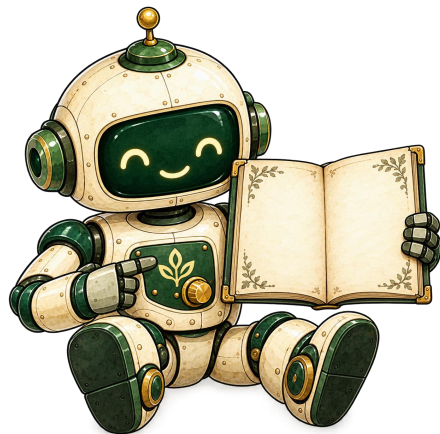
$4 + 5 = 9; 9 \times 3 = 27.$

Answer: $(4 + 5) \times 3$

The example shows the steps and the answer.



A small set of worked examples can matter



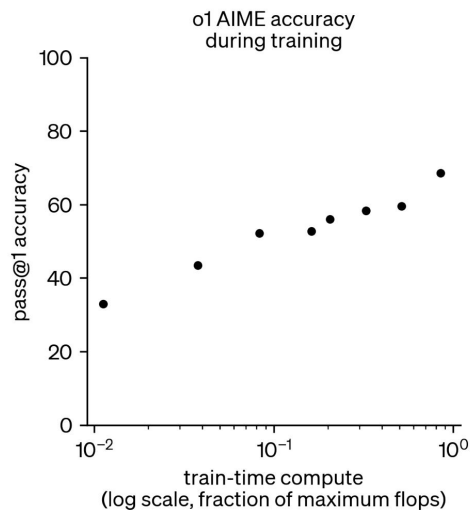
1,000

selected worked examples

s1 · Stanford-led open research

s1 used SFT on 1,000 selected reasoning examples. Data quality mattered.

o1: more RL training improved accuracy

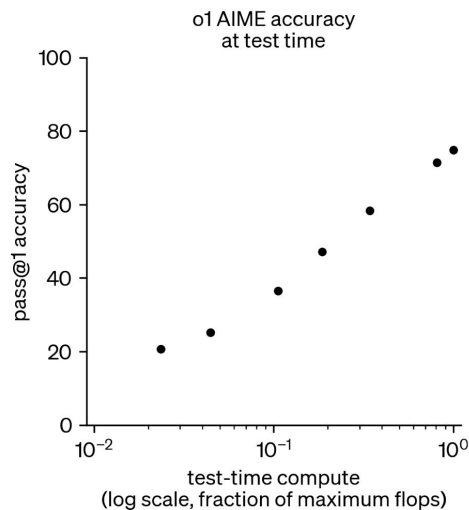


Weights change

**More training
for future questions**

Original measured results. Compute is relative and plotted on a log scale.

o1: more inference effort also helped



Weights stay fixed

**More work
on this question**

The model uses more computation on this question while its weights stay fixed.



How much reasoning does this question need?

$$2 + 3 = ?$$

$$3, 4, 5 \rightarrow 27$$

A short answer is enough

Try a combination and
check it

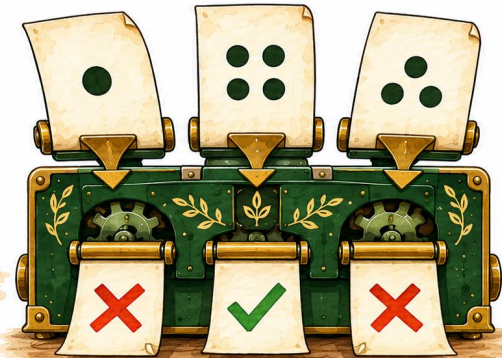
Longer text alone is not evidence of better reasoning.

Extra work costs time and tokens. Check whether it improves the answer.

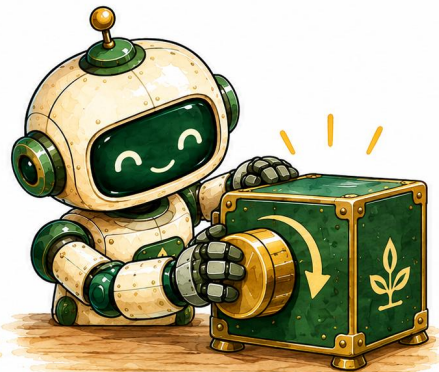
R1-Zero learned from checked outcomes



Generate attempts



Check the answers



Update weights

The next section opens this reward-guided learning loop: reinforcement learning.



R1 combined SFT and RL

01 Cold-start SFT

A few readable worked examples

02 Reasoning RL

Reward checked solutions

03 New SFT round

Keep good attempts; add other tasks

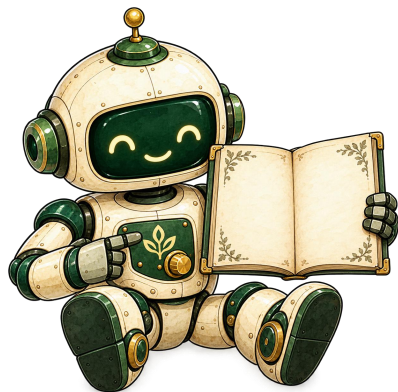
04 Broad RL

Improve usefulness across tasks

The released R1 used several training stages. It was not the R1-Zero recipe.

A smaller model can learn from R1 examples

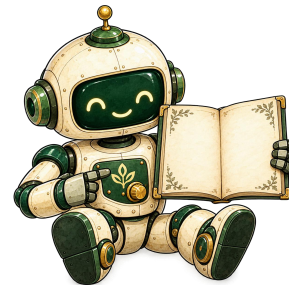
R1



Worked solutions



Smaller model



SFT on those examples

R1 supplies reasoning demonstrations. Smaller models learn through SFT.



The teacher generates several worked solutions

Use 3, 4 and 5 once each. Make 27.

Same prompt, different samples

Attempt A $\longrightarrow 3 + 4 + 5 = 12$

Attempt B $\longrightarrow 3 \times (4 + 5) = 27$

The trained teacher generates tokens. Its weights stay fixed.



Only checked solutions enter the dataset

$$3 + 4 + 5 = 12$$

REJECT

$$3 \times (4 + 5) = 27$$

KEEP

Check the allowed numbers, the arithmetic and the explanation.



The saved sample includes the working

```
{  
  "user": "Use 3, 4, 5 once. Make 27.",  
  "assistant": "4 + 5 = 9. Then 3 * 9 = 27."  
}
```

Prompt + checked worked solution = one SFT example.



The student learns the teacher's answer tokens

Training target

$4 + 5 = 9$. Then $3 \times 9 = 27$.

Predict tokens $\longrightarrow$ Compare with target $\longrightarrow$ Update student

Teacher: fixed. Student: SFT updates. Evaluation: unseen questions.



Can we learn by trying and checking?

Demonstrations help.
Outcomes add another signal.





04

RL

SO FAR

We can score a proposed solution.

PART 4 OF 6



OUR NEXT QUESTION

How can those scores improve future attempts?

Learning from consequences



Notice the cue



Try an action



Receive a treat

A useful analogy: animals can learn actions through their consequences.

The same scene, named in RL

Observation

Hand cue



Action

Sit



Reward

Treat



The full state includes the dog, trainer and surroundings. The dog observes only part.

Which action becomes more likely?



Jump: no treat

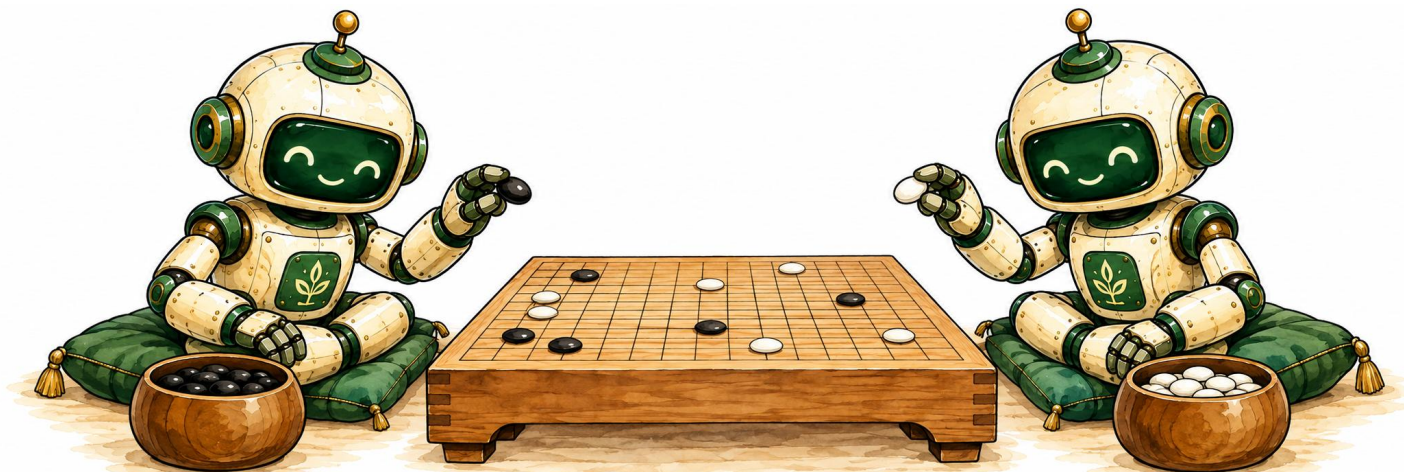


Sit: treat

After repeated rewarded practice, the cue can make sitting more likely.

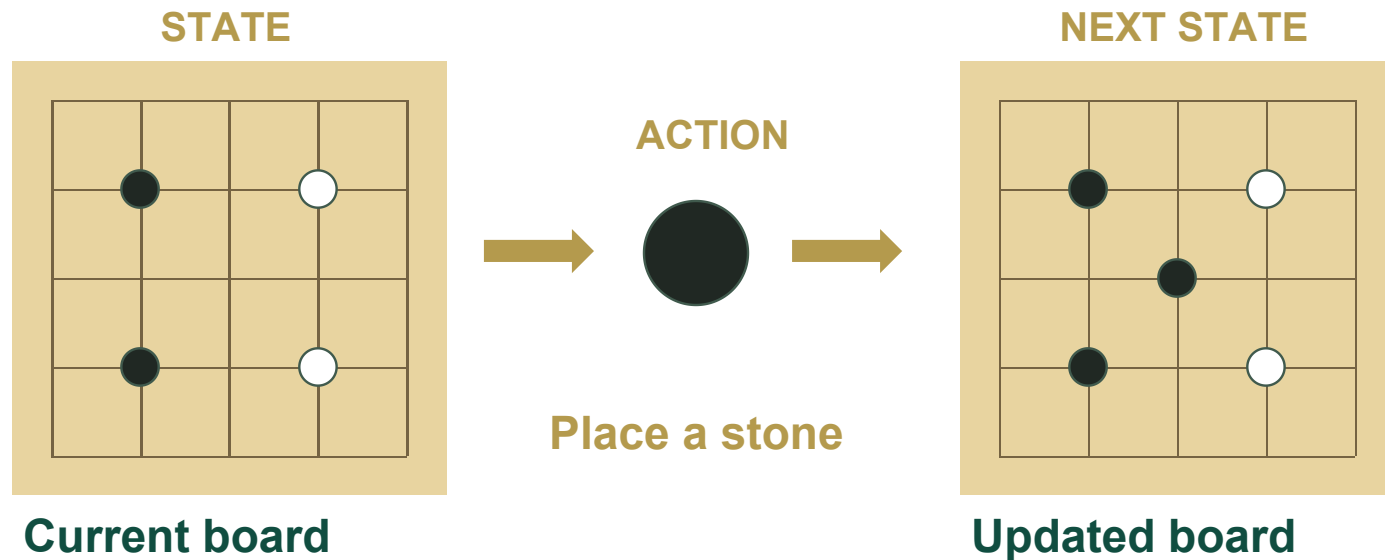
What if the reward comes much later?

A dog can receive a treat immediately. A game may take hundreds of moves.



Go: black and white take turns. The final result provides feedback.

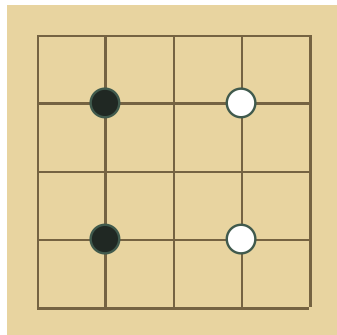
The same RL ideas fit a game



Agent: the player. Environment: rules, board and opponent.

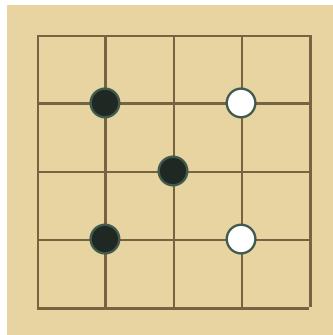
We won. Which moves helped?

Credit Assignment: connect an outcome to earlier decisions.



Early move

...



Later move

...

WIN

Game ends

Which earlier choices deserve reinforcement?

One win does not prove that every move was good.



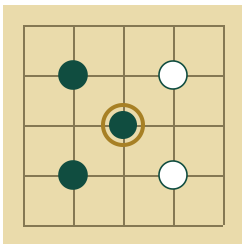
A final result can train earlier decisions



Across many games, reinforce moves with better future returns.

Many games help assign credit

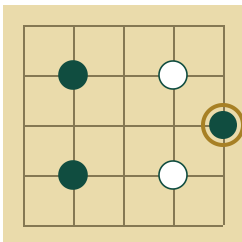
Same board. Try each move, then finish the game.



Move A



2 / 5 wins



Move B



4 / 5 wins

Record: board + chosen move + final result



Better outcomes change future choices

Compare each move with the current expectation.

Expected win rate: 60%

Move A $40\% - 60\% = -20\%$

Move B $80\% - 60\% = +20\%$

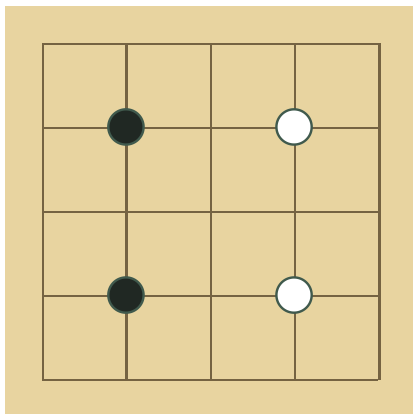
Chance of choosing B



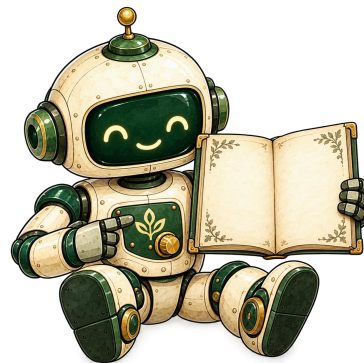
Repeat across many games. One win is noisy evidence.

Illustrative estimates and update. Both moves initially have a 50% selection probability.

AlphaGo first learned from human games



Black at centre

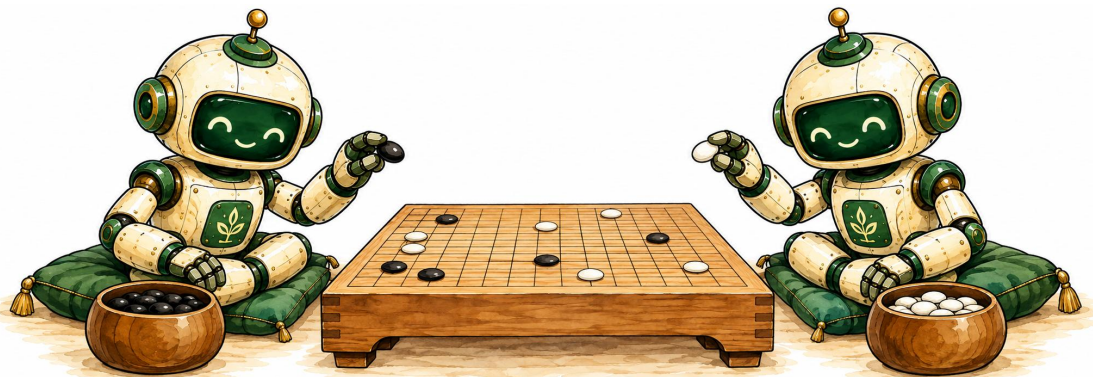


Predict the expert's move

Then self-play RL improved its move choices.

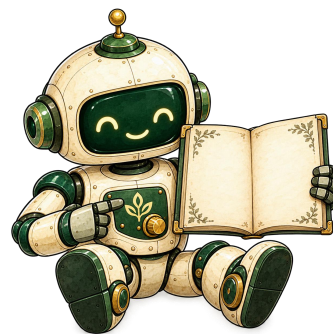
Can the player create its own practice?

AlphaGo Zero: no human game demonstrations.



Play both sides

Self-play produces positions, search targets and final outcomes.



Update the network



A maths question gives us a clear result

\$12 per hour

10 min

10 min

10 min

10 min

10 min

10 min

50 minutes worked

Pay = ?



The model can try different calculations

\$12/hour for 50 minutes

$$12 \times 50$$

600

Reward 0

$$12 \times 50 / 60$$

10

Reward 1

$$12 + 50$$

62

Reward 0



The checker compares with the known answer

GROUND TRUTH

\$10



```
answer = extract_final(reply)
reward = 1 if answer == 10 else 0
```

A correct final number earns 1 in this example.



RL makes rewarded answers more likely

\$12/hour for 50 minutes. Correct answer: \$10.

Before RL

5%



● Correct (\$10)

● Other answers



RL training

After RL

60%



Each dot = 5%

Repeated training makes success more likely. It can still make mistakes.



Where does the feedback come from?

People

A or B?

RLHF

Reinforcement Learning
from Human Feedback

Labels take time

AI judge

A or B?

RLAIF

Reinforcement Learning
from AI Feedback

Judges can be wrong

Rule or test

Pass or fail?

RLVR

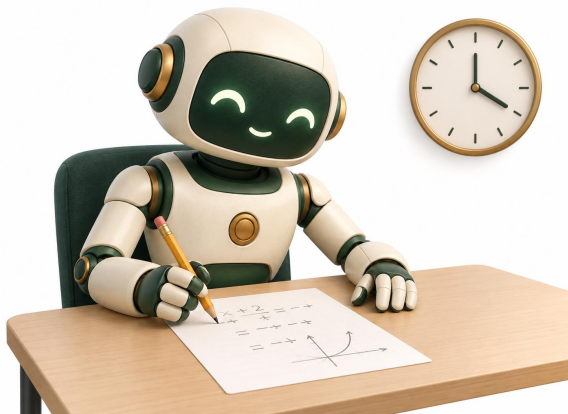
Reinforcement Learning
with Verifiable Rewards

Only checks encoded rules

The task decides which feedback is useful. These routes can be combined.

A rollout is one complete attempt

\$12/hour for 50 minutes



$12 / 60 = 0.20$ per minute

$0.20 * 50 = 10$

Final answer: \$10

One complete solution = one rollout. No weight update yet.



A rollout batch collects several attempts

2 questions × 3 attempts = 6 rollouts

Q1: \$12/h, 50 min

600

10

62

Q2: \$18/h, 40 min

12

720

58

Each row is one comparison group. Both rows form this rollout batch.



Collected attempts become training mini-batches

**6 scored
rollouts**



**Mini-batches
3 + 3**



**Gradient
updates**

Batch sizes describe what we process together, not a new learning method.



A coding rollout can span many tool calls

Example issue: sorting an empty list crashes.



Read file

sort.py

Edit code

if not items: return []

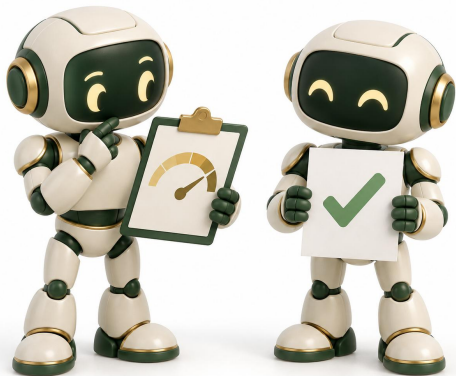
Run tests

[] → [] ✓ [2,1] → [1,2] ✓

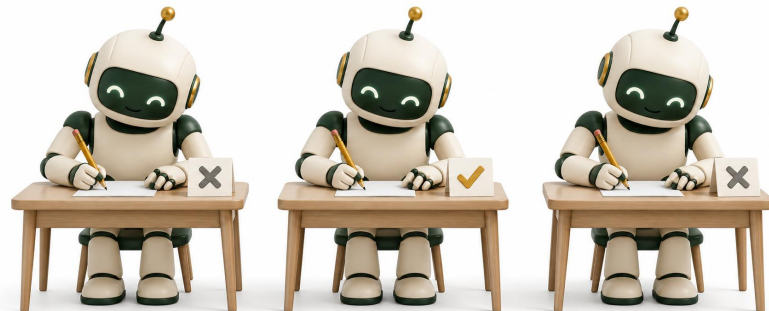
Many tool calls, one rollout. Tests supply the final score.

PPO and GRPO change the comparison point

PPO: result versus expectation



GRPO: compare same-question attempts



Both can train the same task. They estimate “better” differently.



Correctness and explanation quality differ

Answer checker

Human preference

**Is the final
number 10?**

**Which explanation
is easier to follow?**



Both answers are correct. Which explains it better?

Request: “Explain the pay calculation briefly.”

“\$10.”

“ $\$12 / 60 \times 50 = \$10.$ ”

Here the user asked for a short explanation.



A preference pair includes the same question

Less preferred

“\$10.”

Preferred

“50 minutes is $\frac{5}{6}$ hour.
 $\$12 \times \frac{5}{6} = \$10.$ ”

The comparison teaches how to explain, beyond the final number.

Human choices teach a scorer



Later, the trained reward model scores new attempts automatically.



One round of RL practice

01 `attempts = generate(model, request, n=3)`

Try several answers

02 `rewards = score(attempts)`

Check each attempt

03 `update_weights(attempts, rewards)`

Learn from the feedback

04 `repeat_with_updated_model()`

Generate fresh attempts

One reward guides an update. How large should that update be?



PPO limits how sharply the policy changes

PPO · Proximal Policy Optimization

Current policy



After one update



C received a better reward

Increase its likelihood carefully

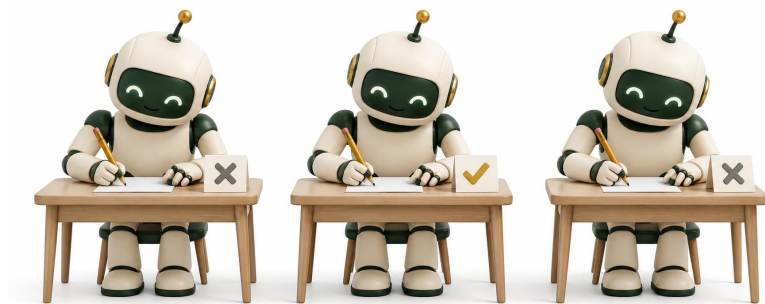
Schematic likelihoods

Clipping discourages overly large policy changes. It is not a hard guarantee.



GRPO compares attempts on the same question

Group Relative Policy Optimization



600 $\rightarrow$ 0

10 $\rightarrow$ 1

62 $\rightarrow$ 0

Same model, same question, three sampled attempts. Answer $\rightarrow$ reward.



Above the group earns a stronger learning signal

Final answer	Reward – mean	Likelihood
600	0 – 1/3	Decrease
10	1 – 1/3	Increase
62	0 – 1/3	Decrease

Equal scores give no within-group preference signal.



Choose the reward, then the update method

Where does the reward come from?

Human preference

RLHF

A program checks the result

RLVR

How do we update the policy?

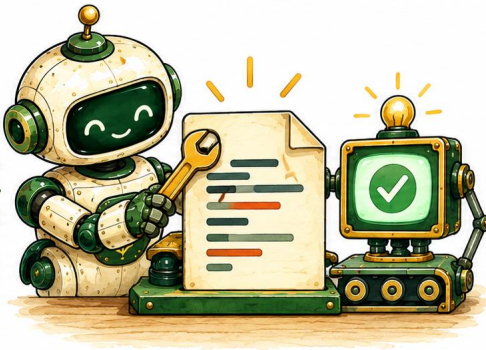
PPO / GRPO

RLHF or RLVR tells us where feedback comes from. PPO or GRPO tells us how to update.

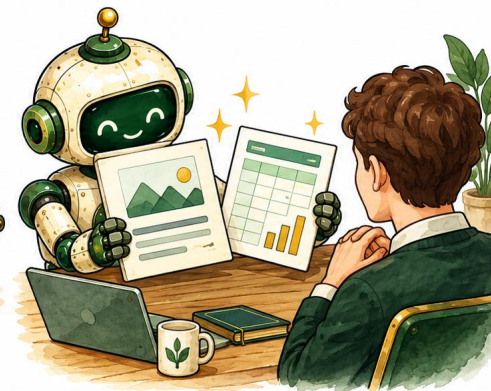
What does a benchmark ask the assistant to do?



Terminal tasks



Software changes



Professional deliverables

What is the task? What counts as success?



Terminal-Bench checks work inside a computer

TASK

Fix the project so its command can run.

BEFORE

app start → missing dependency

VERIFIER

app start → service responds ✓

Run the task, then check the computer's final state.



SWE-bench checks a repository repair

ISSUE

“Sorting must also handle an empty list.”

ACTION

Read the repository. Edit the function.

CHECK

Empty input ✓ Existing behaviour ✓

The code must pass the tests.



GDPval evaluates professional work products

REQUEST

Create a one-page stage plan for a band.

INPUT

Musicians, microphones and equipment list

OUTPUT

Stage layout + input/output lists

Experts compare the finished work.



05

Hands-on

SO FAR

We have a training idea. A small task will make every step visible.

PART 5 OF 6



OUR NEXT QUESTION

How does NanoChat put these ideas into code?



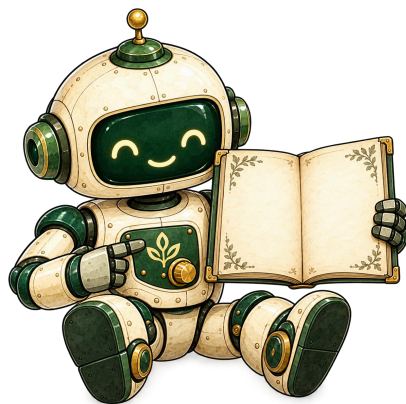
What goes into a training recipe?

We have the idea.
What do we need to run it?



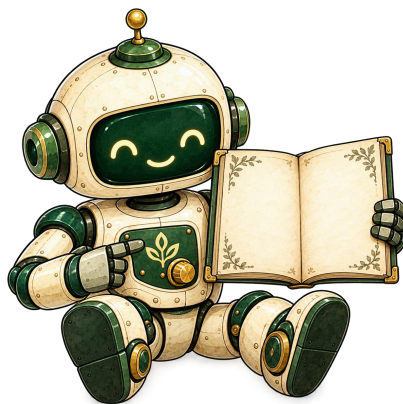
NanoChat supplies the parts of the training recipe

Base model



`base_train.py`

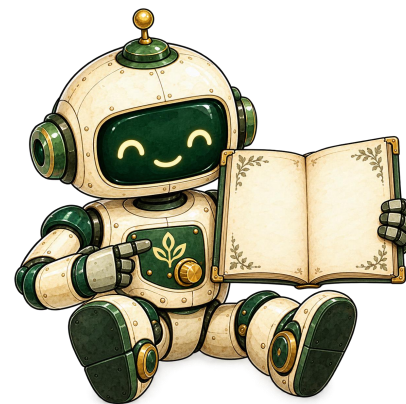
Learn demonstrations



`chat_sft.py`



Learn from attempts



`chat_rl.py`

Classroom: inspect examples and the checker. Full training needs suitable compute.

We start with a maths task so the target and reward are easy to inspect.



A small maths question gives us a clear target

Weng earns \$12 per hour. She works for 50 minutes.

How much does she earn?



GSM8K example from the repository. Correct answer: \$10.



SFT shows a solution. RL checks an attempt

Same question: \$12/hour for 50 minutes

SFT

$12 \div 60 = 0.2$
 $0.2 \times 50 = 10$
10

Learn the demonstrated answer

RL

Model generates
its own solution
and final answer

Check against the stored 10

NanoChat loads the SFT checkpoint before its optional RL stage.



One question produces several attempts

A

$$12 \times 50 \\ = 600$$

600

Reward 0

B

$$12 \div 60 \times 50 \\ = 10$$

10

Reward 1

C

$$50/60 \text{ hour} \times 12 \\ = 10$$

10

Reward 1

D

$$12 + 50 \\ = 62$$

62

Reward 0

Illustrative replies. The real checker compares the final number after ####.



The checker reads the final answer

- | | | |
|----|--|---------------------|
| 01 | <code>predicted = extract_after("####", reply)</code> | Read model answer |
| 02 | <code>expected = extract_after("####", reference)</code> | Read stored answer |
| 03 | <code>reward = 1 if predicted == expected else 0</code> | Compare the strings |

"#### 10"



Reward 1

This reward checks the final number. It does not verify every reasoning step.



Which attempts did better than this group?

Reward 0

-0.5



Reward 1

+0.5



Reward 1

+0.5



Reward 0

-0.5



Mean reward: 0.5

Above average: reinforce. Below average: discourage.

Compare each score with the group average.



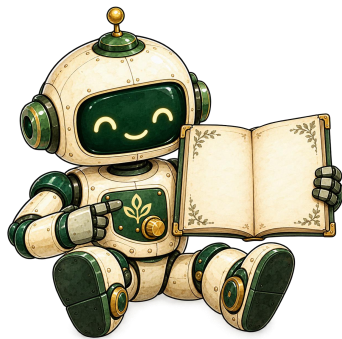
One NanoChat RL update

- | | | |
|-----------|--|------------------------------|
| 01 | <code>attempts = model.generate(question, many=True)</code> | Try several answers |
| 02 | <code>rewards = check_final_answers(attempts)</code> | Score each answer |
| 03 | <code>advantages = rewards - mean(rewards)</code> | Compare within the group |
| 04 | <code>loss = policy_loss(attempts, advantages)</code> | Use generated-token scores |
| 05 | <code>loss.backward()</code> | Compute the update direction |
| 06 | <code>optimizer.step()</code> | Update parameters |

Simplified policy-gradient update from NanoChat. Only generated tokens contribute.

Does RL help on held-out questions?

Before RL

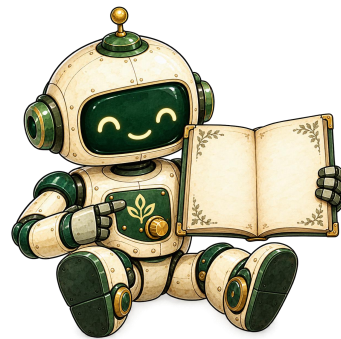


SFT checkpoint

After RL

Same test set

pass@1
first attempt
succeeds



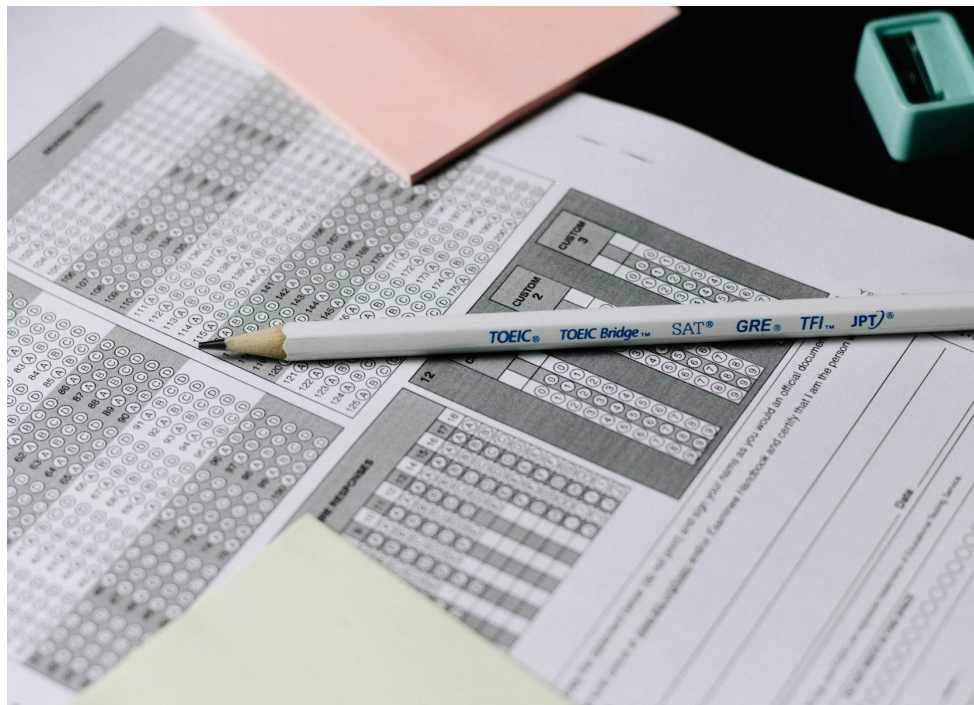
RL checkpoint

Use the same questions and generation budget before and after training.



Right answer. Right reasoning?

What can a checker see
if it only reads the final answer?





This checker reads the number, not the reasoning

\$12 per hour, for 50 minutes

$$12 \div 60 \times 50 = 10$$

10

Reward 1

Sound calculation

$$12 + 50 = 10$$

10

Reward 1

Wrong calculation

The final answer can pass even when the working is wrong.

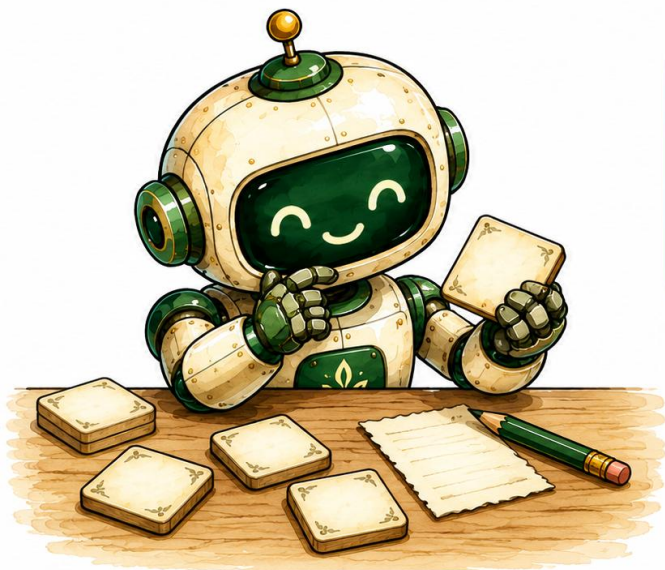
TinyZero: the same idea in a number game

TinyZero / Countdown



Countdown checks numbers and target. The original TinyZero experiment uses PPO.

Can you make 20?



TinyZero / Countdown: use 2, 4 and 6 once each. What should the checker accept?



Only B uses every number and reaches 20

A

`<answer>`
20
`</answer>`

0.1

B

`<answer>`
 $(6+4)*2$
`</answer>`

1

C

`<answer>`
 $6+4+2$
`</answer>`

0.1

B solves the puzzle. A and C receive only the small extractability reward.



香港大學
THE UNIVERSITY OF HONG KONG

LLM FOUNDATION · LECTURE 3

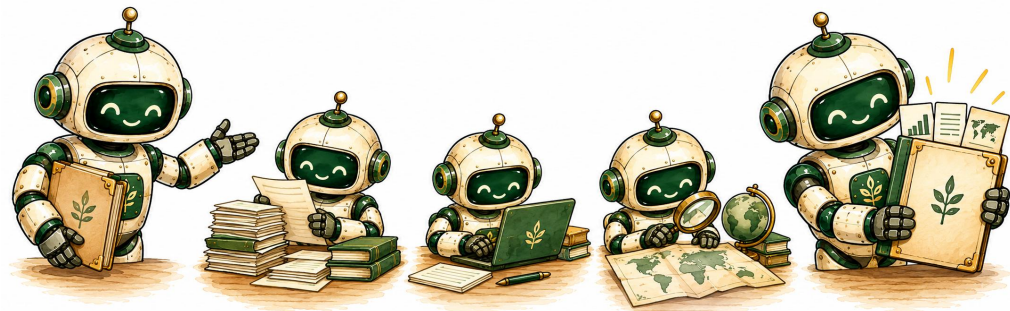
06

Insights

SO FAR

We can train and test a small task.
Larger systems add new tradeoffs.

PART 6 OF 6



OUR NEXT QUESTION

What changes when models and tasks become larger?



Post-training recipes keep expanding

2024



OpenAI o1

RL for useful reasoning

2025



R1 / Qwen3

Checked answers and
tool use

2026



GLM / Kimi / V4.1

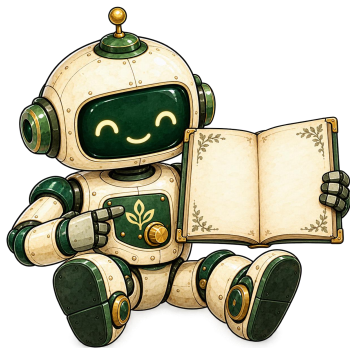
Longer tasks and agent
coordination

These are different research recipes, not stages every model must follow.

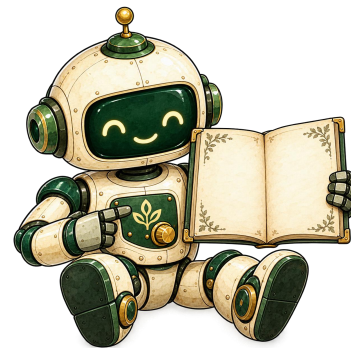
GLM-5.3 continued from the same base

Same base model

Continued post-training



GLM-5.2



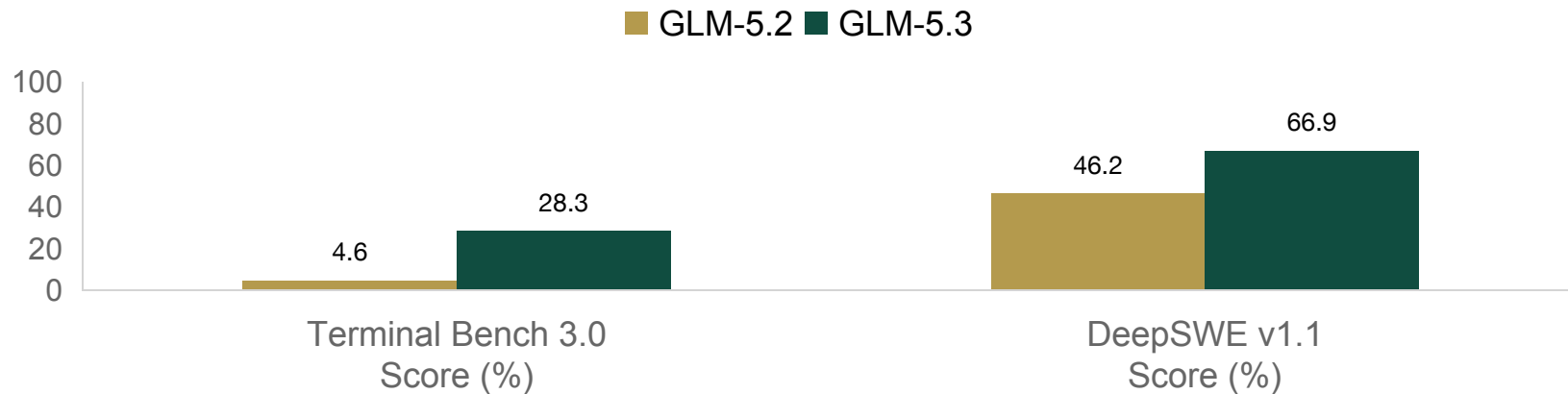
GLM-5.3

Z.ai attributes the gains to post-training. This does not isolate one RL algorithm.



Could it finish more executable tasks?

Terminal tasks and software engineering tasks



Publisher-reported scores. Same base; details of the training recipe and tests still matter.

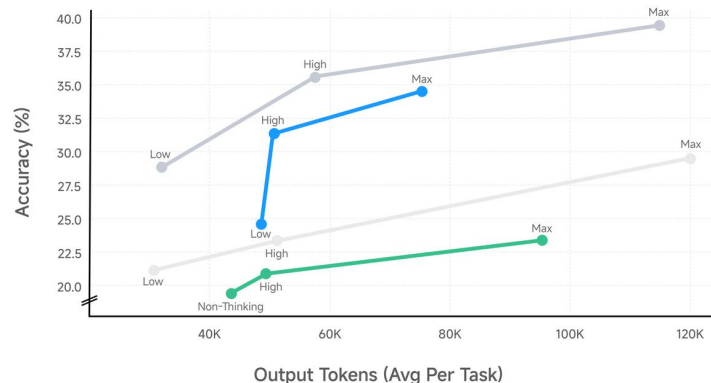
Better results can use fewer tokens

Agentic Coding Performance by Effort Level

Z.ai Code Bench v1.0, evaluated on Claude Code 2.1.207



GLM-5.3 GLM-5.2 Claude Fable 5 Claude Opus 4.8



Higher accuracy

23.4% to 34.5%

Fewer output tokens

~96K to ~75K

Z.ai Code Bench v1.0, max effort. Private benchmark, publisher-reported results.

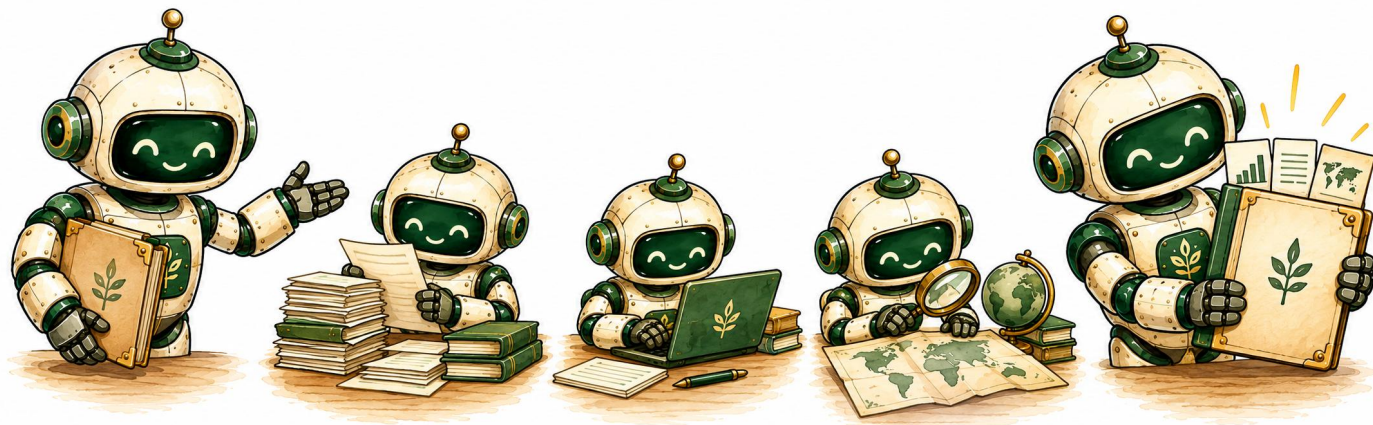


Would extra hands finish it sooner?

Find three course readings.
Could separate searches help?



Kimi K2.5 explored an agent swarm



Delegate

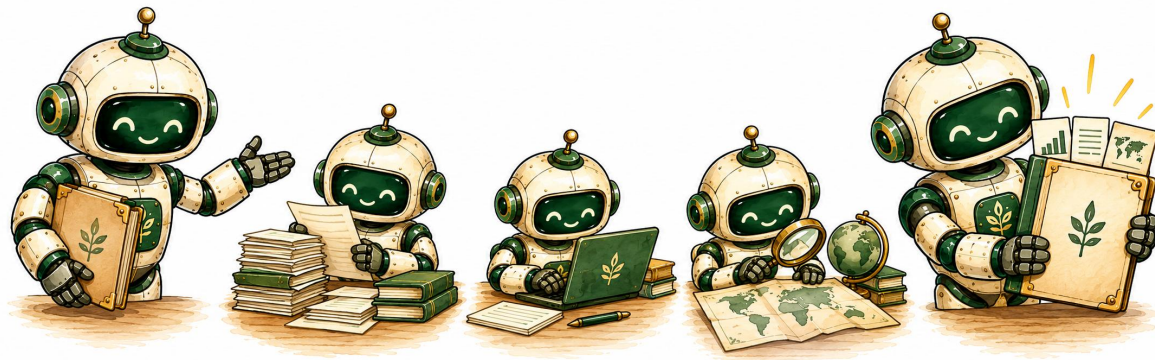
Search in parallel

Combine

The coordinator can assign separate searches, then combine the readings.

PARL trains the coordinator's decisions

Parallel-Agent Reinforcement Learning

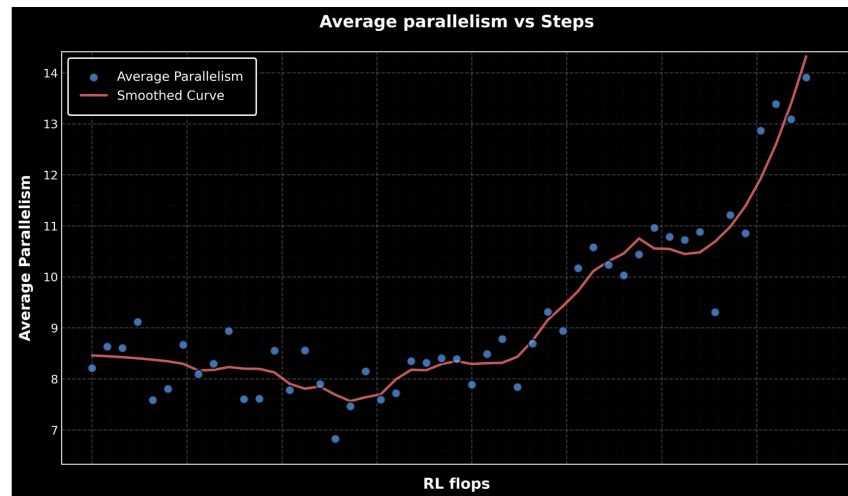
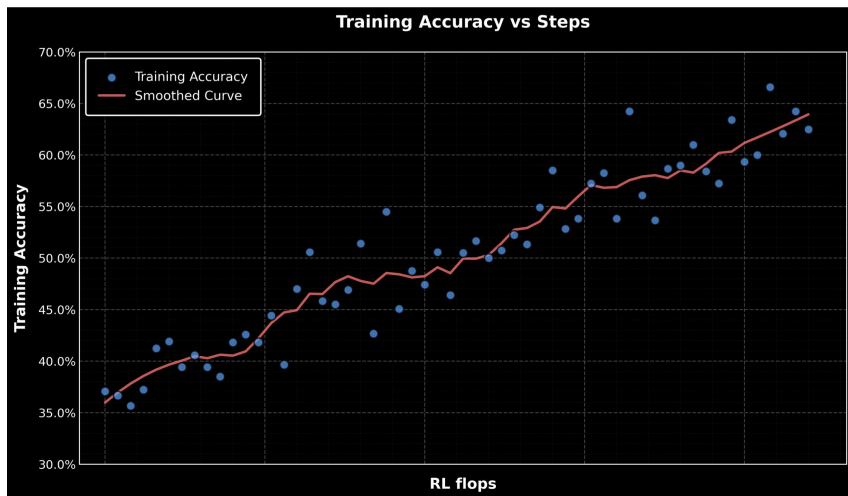


Coordinator weights update

Subagent weights stay fixed

In the reported setup, subagents stay frozen while the orchestrator learns.

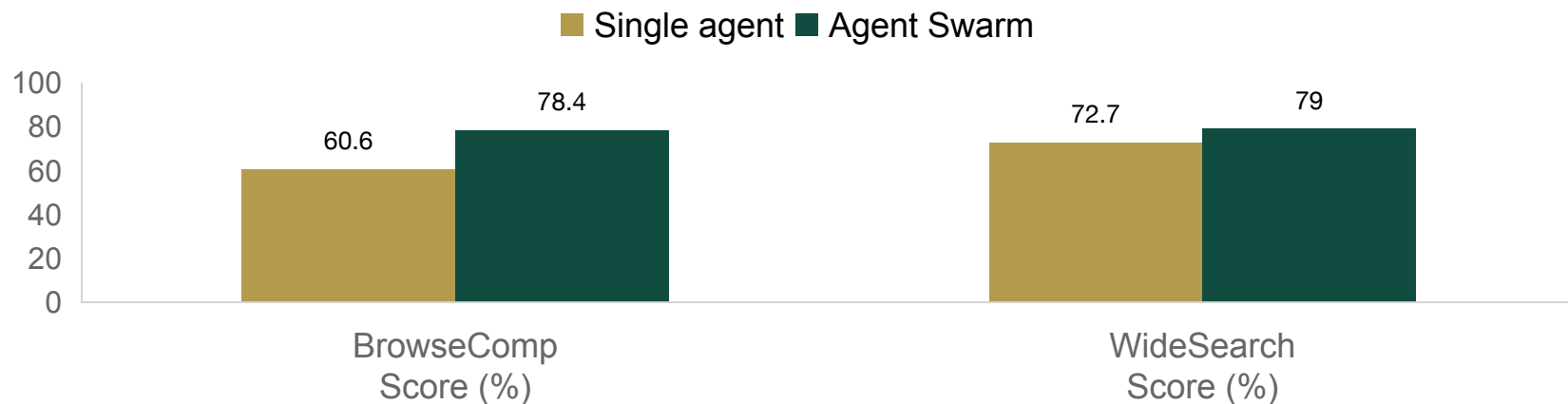
The coordinator improves during RL training



Original reported training curves. They describe this experiment, not every agent system.

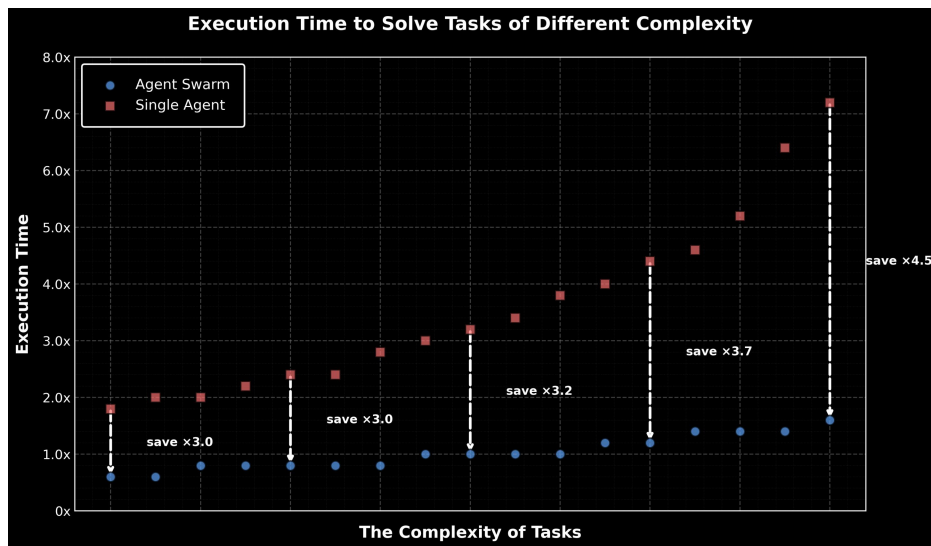


Did the extra agents help the task?



Single-agent and swarm execution use different budgets. This is a system-level comparison.

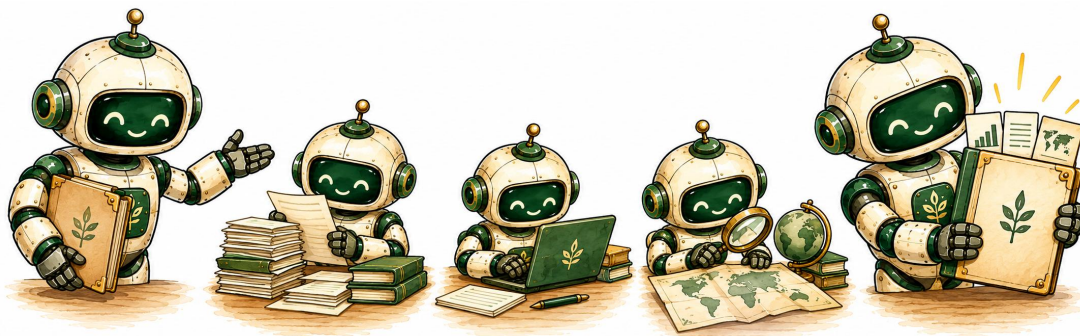
Parallel work can reduce elapsed time



These are publisher-reported results for a specific setup, not a universal speedup.

Extra workers can duplicate the same work

Three workers return the same paper.



More activity

Little extra useful information

Check success, time and total cost. More agents can cost more.



Learn, check, improve

Maths

**Correct answer
and valid working**

Tool tasks

**Right action
and arguments**

Slides

**Relevant pictures
and readable
layout**

Learn from demonstrations and scores. Test on unseen tasks.